1 Restaurant Website Content Extraction

1.0.0.1 1. Overview

To get the HTML content and images from restaurant websites, we utilized a relatively simple spider using Scrapy. This was gathered using the OpenStreetMap data, with the documentation for that here.

The raw HTML was gathered for future general use rather than any particular function. This HTML has had some unneeded content removed to save on storage and to increase readability. The images were gathered primarily for the purpose of recognizing and classifying cousine types of the restaurant based on their foods.

Ultimately this serves the purpose of giving us information on the websites of Switzerland, as well as their images to be used during classification later in this project.

1.0.0.2 2. Requirements

Relies on `restaurants_filtered.json` to parse out the list of available websites on *OpenStreetMap.* While these URLs are valid in format, many are no longer active, point to non-restaurant domains, or are inaccessible to scrapers. Sites may:

- Have expired domains
- Block bots entirely
- Redirect to unrelated businesses
- Not be restaurants at all

1.0.0.3 3. Spider Behavior

The spider first scrapes the html and images off of the main page that is found from the *OpenStreetMap* data. After this, the spider will recursively crawl through the child sites on that domain. The spider handles requests rapidly to allow for managing the many websites in a timely manner. However, there are two caveats to this crawling:

- Only 20 internal links are followed on any given page. There are sites that contain an unreasonable number of sub-pages, and trying to navigate through those stops the spider for far too long, and seldom provides greater insight on the restaurant.

- There are a short list of domains that we have opted to skip. These are sites that we ran into that cause problems that are not actually restaurants. These are businesses that range from movie theaters to research centers that have an attached cafe, which pull in unneeded information.

1.0.0.4 4. File/Folder Structure

For saving the information of the sites, we opted to save them directly in our files rather than setting up a database. This was primarily for the ease of access to images, as well as the scale of the information, which may have been difficult to store in a database. The file structure for storage is as follows:

```
scraped-data/
     {element_id}/
           text-content/
           images/
           screenshot/
scrape_status.csv
```

The content of these folders and files:

- scraped-data: Contains a list of folders that each contain the HTML and images from scraped restaurants
- {element_id}: The ID of the restaurant provided by *OpenStreetMap*
- text-content: A list of HTML files that contain all non-script, non-style HTML from the page
- images: Contains all of the image files present in that domain
- screenshot: A placeholder file for screenshots, which may be removed if we decide screenshots are unnecessary
- scrape_status.csv: Contains information that helps us maintain the status of what has been scraped from a website, as well as whether or not there is a URL provided

1.0.0.5 5. CSV Status Tracker

This .csv file tracks the status of scraped restaurant websites. The schema is as follows:

Column Name	Type	Description
`id`	string	The unique OpenStreetMap ID for the restaurant
`has_website`	boolean	Whether the original JSON entry included a valid website URL

Column Name	Type	Description
`is_scraped`	boolean	Whether the website was successfully scraped (HTML and image data gathered)
`is_filtered`	boolean	Whether the site was filtered out due to lacking food-related images

This is updated in the spider such that newly scraped websites set *is_scraped* to true.

1.0.0.6 6. Limitations

There are some limitations that are inherent in scraping this much information from this many sites. The biggest issues that we ran into are as follows:

- JavaScript Heavy Sites: Sites that utilize a large amount JavaScript are not rendered prior to being scraped. This leads to some sites only having partial information.
- Sites With No Relevant Images: Some sites contain no images, or contain no images that relate to the food they serve, which make the information scraped from them not useful.
- Non Scrape-Friendly Sites: Some sites block scraping altogether or prevent reading of content while scraping. This leads to some sites that work and have images on them when navigating with a browser, but have no images downloaded.

1.0.0.7 7. Restaurants without a provided Link

Some restaurants in the *OpenStreetMap* data do not have a provided link to their website. For that we developed a script that uses **Nominatim** from Geopy and **DuckDuckGo_Search** to find the website of the restaurant based on its name and location. It uses the coordinates, which are always provided to get the name of the city. Then together with the name and the city it searches for the website. It may not be perfect, but is sufficient for our purposes.